

Relatório 7: Desempenho de Matrizes em Cache

João Bruno dos Santos, João Lucas Medina Kormann

Universidade Federal de Santa Catarina

II. RESULTADOS OBTIDOS

I. INTRODUÇÃO

Este relatório tem o objetivo de apresentar a metodologia aplicada para a resolução dos problemas do Laboratório 7 da disciplina Organização de Computadores I, ministrada pelo professor Marcelo Daniel Berejuck, utilizando a linguagem Assembly para a arquitetura MIPS (Microprocessor without Interlocked Pipelined Stages), o simulador MARS (MIPS Assembler and Runtime Simulator).

O intuito principal é analisar o comportamento do cache de dados durante a execução de diferentes implementações de manipulação de matrizes, previamente desenvolvidas no Laboratório 5, sob diversos parâmetros de configuração de cache. Para isso, foi empregada a ferramenta Data Cache Simulator, permitindo observar e comparar métricas como o Cache Miss Rate em diferentes tamanhos e organizações de cache, além de avaliar o impacto de diferentes políticas de mapeamento e substituição. Através dessa análise, busca-se compreender como o acesso à memória influencia o desempenho do sistema.

Tabela 1 - Resultados com preenchimento de matriz linha a linha, com política de alocação Direct Mapping

No of Blocks	Cache block size (words)	Memory access count	Cache Hit Count	Cache Miss Count	Cache Hit Rate
8	4	1329	1208	121	91%
8	8	1329	1236	93	93%
16	8	1329	1266	63	95%
16	16	1329	1231	48	96%
16	32	1329	1320	9	99%

Tabela 2 - Resultados com preenchimento de matriz linha a linha, com política de alocação Fully Associative

No of Blocks	Cache block size (words)	Memory access count	Cache Hit Count	Cache Miss Count	Cache Hit Rate
8	4	1329	1264	65	95%
8	8	1329	1296	33	98%
16	8	1329	1296	33	98%
16	16	1329	1312	17	99%
16	32	1329	1320	9	99%

Tabela 3 - Resultados com preenchimento de matriz linha a linha, com política de alocação 2-way Set Associative

No of Blocks	Cache block size (words)	Memory access count	Cache Hit Count	Cache Miss Count	Cache Hit Rate
8	4	1329	1264	65	95%
8	8	1329	1296	33	98%
16	8	1329	1296	33	98%
16	16	1329	1312	17	99%
16	32	1329	1320	9	99%

Tabela 4 - Resultados com preenchimento de matriz coluna a coluna, com política de alocação Direct Mapping

No of Blocks	Cache block size (words)	Memory access count	Cache Hit Count	Cache Miss Count	Cache Hit Rate
8	4	1329	1040	289	78%
8	8	1329	1040	289	78%
16	8	1329	1056	273	79%
16	16	1329	1281	48	96%
16	32	1329	1320	9	99%

Tabela 5 - Resultados com preenchimento de matriz coluna a coluna, com política de alocação Fully Associative

No of Blocks	Cache block size (words)	Memory access count	Cache Hit Count	Cache Miss Count	Cache Hit Rate
8	4	1329	1072	257	81%
8	8	1329	1072	257	81%
16	8	1329	1072	257	81%
16	16	1329	1072	257	81%
16	32	1329	1320	9	99%

Tabela 6 - Resultados com preenchimento de matriz coluna a coluna, com política de alocação 2-way Set Associative

No of Blocks	Cache block size (words)	Memory access count	Cache Hit Count	Cache Miss Count	Cache Hit Rate
8	4	1329	1072	257	81%
8	8	1329	1072	257	81%
16	8	1329	1072	257	81%
16	16	1329	1282	96	96%
16	32	1329	1320	9	99%

Gráfico 1 - Cache Hit Rate x No of Blocks-Block Size para matriz linha a linha

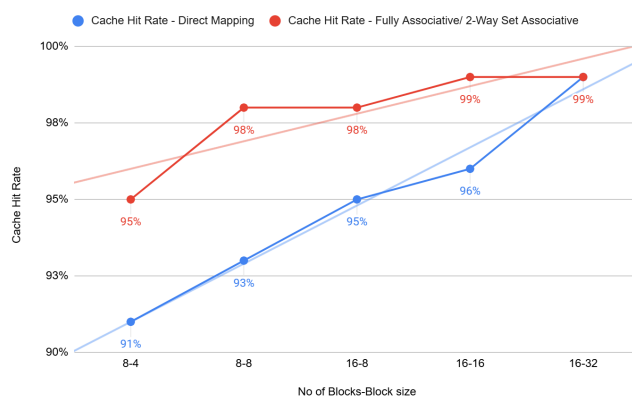
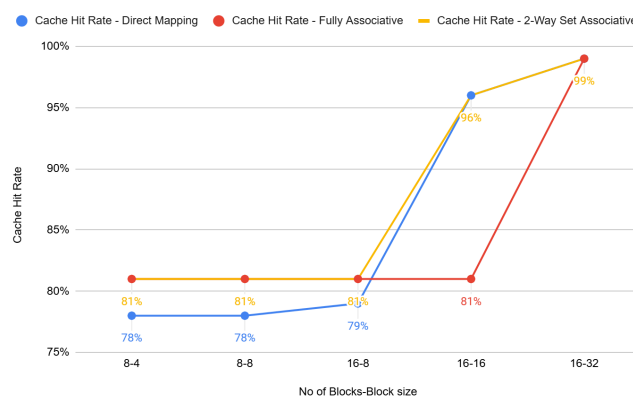


Gráfico 2 - Cache Hit Rate x No of Blocks-Block Size para matriz coluna a coluna



III. DISCUSSÃO DOS RESULTADOS

A. COMPARAÇÃO DO PADRÃO DE ACESSO

No preenchimento linha a linha, a taxa de acerto do cache foi consistentemente maior em todos os cenários testados. Isso ocorre porque o acesso sequencial aos elementos da matriz segue a mesma ordem de armazenamento na memória, aproveitando a localidade espacial. Ou seja, quando um elemento é carregado na cache, os próximos elementos acessados já se encontram no mesmo bloco.

Esse comportamento fica evidente nos resultados, onde em configurações com 8 blocos de 4 palavras, a taxa de acerto já ultrapassa 90%, e à medida que o tamanho dos blocos aumenta, o Cache Hit Rate atinge até 99%, praticamente eliminando *misses*. Assim, quanto maior o bloco, maior o aproveitamento da localidade espacial.

Contudo, o preenchimento coluna a coluna apresentou taxas de acerto bem menores nas configurações iniciais. Isso se deve ao fato de que, ao acessar os elementos de uma mesma coluna, é preciso saltar grandes intervalos de endereços entre uma linha e outra, o que quebra a localidade espacial e faz com que praticamente cada acesso recaia em um bloco de memória diferente. Como resultado, caches pequenos e blocos pequenos sofrem alto índice de *misses*.

Porém, à medida que o número e o tamanho dos blocos aumentam, o desempenho do acesso por coluna também melhora, como demonstra o Gráfico 2, indicando que aumentar a capacidade da cache pode mitigar os efeitos negativos de padrões de acesso desfavoráveis. Ainda assim, mesmo com caches maiores, o acesso coluna a coluna permanece menos eficiente do que o acesso linha a linha, reforçando a importância de projetar algoritmos que aproveitem a organização física da memória.

B. INFLUÊNCIA DO TAMANHO E NÚMERO DE BLOCOS

Tanto o número de blocos quanto o tamanho de cada bloco exercem influência significativa no desempenho.

Aumentar o número de blocos reduz a probabilidade de substituição de dados ainda úteis,

diminuindo os conflitos de mapeamento e, consequentemente, a taxa de *misses*.

Já o aumento do tamanho dos blocos amplia o benefício da localidade espacial, pois mais palavras consecutivas da memória principal são carregadas em uma única transferência, aumentando a probabilidade de que futuros acessos encontrem os dados desejados já na cache.

Essa tendência é observada em todas as políticas de mapeamento, os cenários com 16 blocos de 32 palavras apresentaram desempenho praticamente ideal, com Cache Hit Rate próximo de 99% para ambas as abordagens (linha e coluna).

C. POLÍTICAS DE MAPEAMENTO

A política de mapeamento define como os blocos da memória principal são armazenados na cache e influencia diretamente o número de conflitos de mapeamento e a taxa de acertos. Nos testes, foram usadas três abordagens: Direct Mapping, 2-Way Set Associative e Fully Associative, nas quais os resultados demonstram diferenças significativas em caches menores.

No Direct Mapping, cada bloco da memória é mapeado para apenas uma posição específica na cache. Contudo, se dois endereços diferentes disputam o mesmo bloco de cache, um deles é substituído mesmo que ainda seja necessário. Esse comportamento foi visível nos resultados: no preenchimento linha a linha, o Cache Hit Rate variou de 91% (8 blocos de 4 words) a 99% (16 blocos de 32 words). Já no acesso coluna a coluna, as taxas iniciais foram bem menores, próximas de 78%, refletindo maior quantidade de *misses* devido à quebra de localidade e à limitação de mapeamento.

A política 2-Way Set Associative divide a cache em conjuntos com dois blocos por conjunto, permitindo que cada endereço de memória seja armazenado em qualquer um dos dois. Essa flexibilidade reduz a ocorrência de conflitos, melhorando o aproveitamento da cache. Nos resultados, as taxas de acerto foram semelhantes às do Fully Associative, alcançando 95% a 99% no caso linha a linha e cerca de 81% no caso coluna a coluna, mesmo em caches menores. Isso demonstra

que aumentar a associatividade melhora o desempenho.

Por fim, no Fully Associative, qualquer bloco de memória pode ocupar qualquer posição na cache. Essa política elimina completamente os conflitos de mapeamento, resultando nas melhores taxas de acerto observadas. No acesso linha a linha, a taxa chegou rapidamente a 99% em quase todas as configurações, e mesmo no acesso coluna a coluna, manteve-se em torno de 81% nas caches menores e 99% nas maiores.

De modo geral, observa-se que políticas mais associativas proporcionam maior aproveitamento da cache, principalmente em configurações pequenas, enquanto em caches maiores as diferenças entre as políticas diminuem. Assim, o Direct Mapping é mais limitado, mas eficiente em hardware, o 2-Way Set Associative representa um bom equilíbrio entre custo e desempenho e o Fully Associative alcança o melhor resultado, ao custo de maior complexidade de implementação.